



The Knuth-Morris-Pratt (KMP) Algorithm

PPT PRESENTED BY

NAME : Srinivas Rao Tammireddy

ROLL NO : 257Y1D5805

YEAR : I Year I Semester

ACADEMIC YEAR : 2025 - 2026

The Pattern Matching Problem

Pattern Matching seeks all occurrences of a pattern string P of length m within a text T of length n .

The Naive Search Algorithm

- **How it works:** Aligns the pattern P at the start of text T . Compares character by character.
- **Mismatch action:** Shift the pattern by exactly ONE position. Reset comparison index and start over.
- **Worst-case Complexity:** $O(n \cdot m)$. Occurs when text and pattern have highly repetitive prefixes (e.g. $T = \text{'AAAAAAB'}$, $P = \text{'AAAB'}$).
- **Limitation:** Wastes information. It forgets character matches that have already occurred, resetting the text pointer backwards.

The KMP Innovation

- **Linear Complexity:** $O(n + m)$ worst-case running time. Preprocesses the pattern in $O(m)$ time and searches in $O(n)$ time.
- **No Text Backtracking:** The index scanning the text T NEVER goes backward. We only move the pattern index forward using prior match data.
- **Key Idea:** When a mismatch occurs, we know the prefix of the pattern matched up to that point. We look at the pattern itself to determine where to shift, skipping useless alignments.

The Prefix Function (π -table / Failure Function)

The π -table stores the lengths of the longest proper prefix that is also a suffix of the pattern:

Mathematical Definition

- **$\pi[q]$:** The length of the longest proper prefix of pattern $P[0..q]$ that is also a suffix of $P[0..q]$.
- **Proper Prefix:** A prefix of string S that is not equal to S itself.
- **Purpose:** If a mismatch occurs at $P[q+1]$, we do not start over. The value $\pi[q]$ tells us the length of the matching prefix that is already aligned, allowing us to resume search at index $P[\pi[q]]$.
- **Preprocessing time:** Computed in $O(m)$ time using dynamic programming.

Example: $P = \text{'ABABC'}$

i	P[i]	$\pi[i]$	Longest Prefix/Suffix Match
0	A	0	No proper prefix exists
1	B	0	No prefix-suffix match for 'AB'
2	A	1	Prefix 'A' matches suffix 'A'
3	B	2	Prefix 'AB' matches suffix 'AB'
4	C	0	No match for 'ABABC'

KMP Matcher Algorithm

How the KMP algorithm matches patterns in linear time:

KMP Search Loop Logic

Let text be T of length n , pattern be P of length m . Maintain indices i for T and j for P :

1. **Initialize:** $i = 0$ (text scan pointer), $j = 0$ (pattern match pointer). Compute π -table for P .
2. **Search Loop:** While $i < n$:
 - **If $T[i] == P[j]$:** Characters match! Increment both: $i++$ and $j++$.
 - **If $j == m$:** Pattern found! Report occurrence at index $i - m$. Slide pattern: set $j = \pi[j - 1]$.
 - **If $T[i] != P[j]$ (Mismatch):** Slide pattern without backtracking i :
 - If $j > 0$: set $j = \pi[j - 1]$ (shift pattern to align the longest matching prefix).
 - If $j == 0$: cannot shift. Simply move text pointer forward: $i++$.

Complexity & Correctness Proof

Mathematical analysis of KMP's performance bounds:

Time Complexity Proof

- **Split phase costs:** Preprocessing takes $O(m)$ steps. Searching takes $O(n)$ steps. Total time is $O(n + m)$.
- **Search Cost Analysis (Amortized):** Let's count the number of comparisons. The loop runs at most $2n$ times because:
 - The index i increases in every step (either by matching or by $j == 0$ mismatch). It never decreases. Hence, at most n increments.
 - The index j changes. We increment j when characters match (at most n times). When we reset $j = \pi[j-1]$ on mismatch, j decreases.
 - Since j starts at 0, is incremented at most n times, and can never drop below 0, it can decrease at most n times. Hence, $j = \pi[j-1]$ can be executed at most n times.
- **Total operations $\leq 2n$.**

Correctness and Safety

- **No Missed Matches:** We must prove that KMP never skips a valid alignment of the pattern.
- **Shift Safety Proof:** Suppose we mismatch at $P[j]$. We know $P[0..j-1]$ matches $T[i-j..i-1]$. We shift P by setting $j = \pi[j-1]$.
 - This is the largest shift size where the new alignment could possibly match.
 - Any smaller shift (which corresponds to a longer prefix matching a suffix of $P[0..j-1]$) is checked because $\pi[j-1]$ is defined as the MAXIMUM prefix-suffix overlap.
 - Any intermediate shift cannot match because it would require a suffix of $P[0..j-1]$ to match a prefix of P , which is mathematically impossible since $\pi[j-1]$ is the longest such match.

Applications of KMP in Computer Science

KMP is widely used in systems requiring fast, streaming pattern searching:

Text Editors & IDE Search

Modern editors (like VS Code, Notepad++) use KMP or Boyer-Moore algorithms to implement 'Find' operations, ensuring instantaneous search and replace in large text files.

Intrusion Detection Systems (IDS)

Network packet scanners (like Snort) use KMP to scan live packets against database signatures of malware and cyber threats. Since KMP does not backtrack, it works on streaming network traffic.

Bioinformatics & DNA Alignments

Genomics software uses KMP to search for specific gene signatures and mutations inside massive DNA sequences containing billions of base pairs.

Log Analysis Tools

Tools like grep or log parsers apply KMP to filter out error logs or extract warnings from massive server log files in a single pass.



THANK YOU

Department of Computer Science and Engineering
Marri Laxman Reddy Institute of Technology and Management